



CrewConnect

Senior Capstone: Design Document

Cody Mathews, Joel Moffatt, Liam Coburn, Spencer Perry

Table of Contents

1.1 Executive Summary.....	4
1.2 Background and Technical Requirements.....	5
1.2.A – Similar ideas.....	5
1.2.B – Technologies to Leverage.....	6
1.2.C – Software/Hardware Requirements.....	7
1.3 Requirements Analysis.....	8
1.3.1 System architecture.....	8
1.3.1.1 Client Layer (Presentation).....	8
1.3.1.2. API / Application Layer.....	8
1.3.1.3. Data Layer (Postgres).....	9
1.3.1.4. Geospatial / Mapbox Layer.....	10
1.3.1.5. File / Media Handling.....	11
1.3.1.6. Infrastructure (AWS).....	11
1.3.1.7. Data Flow (End-to-End).....	12
1.3.2 Personnel.....	12
1.3.3 System features.....	14
1.4 Software Engineering Tools and Techniques.....	16
1.4.1 Agile.....	16
1.4.2 Tools.....	16
1.4.2.1 IDEs.....	16
1.4.2.2 Main Libraries.....	16
1.4.2.3 Postman.....	17
1.4.2.4 Testing Plans and Tools.....	17
1.4.2.5 Bug Tracking.....	17
1.4.2.6 Versioning.....	17
1.4.2.7 Issue Tracking.....	17
1.4.3 Code Reviews, e.g., “entire team” vs “single signoff”.....	17
1.4.4 Documentation.....	18
1.4.5 Team Communication.....	18
1.4.6 Team Meetings.....	18
1.5 Timeline.....	18
1.6 Appendix A: UI Sketches.....	25
1.6.1 Web Dashboard Rough Sketch.....	25
1.6.2 Web Home Page Rough Sketch.....	25
1.6.3 Web Dashboard.....	26
1.6.4 Web Safety Overview.....	26

1.6.5 Web Safety Overview - No Sidebar.....	27
1.6.6 Team Member Safety Review - No Sidebar.....	27
1.6.7 Team Member Safety Review.....	28
1.6.8 Web Labor Overview.....	28
1.6.9 Mobile - App Screen.....	29
1.6.10 Mobile Login page.....	29
1.6.11 Laborer Clock In.....	30
1.6.12 Mobile - Observation Screen.....	31
1.6.13 Mobile - Pre Operation Screen.....	31
1.7 Appendix B: Use Cases.....	32
Use Case 1.....	32
Use Case 2.....	33
Use Case 3.....	34
Use Case 4.....	35
Use Case 5.....	36
1.8 Appendix C: Revisions.....	37
1.8.1 - Executive Summary.....	37
1.8.2 - Background and Technical Requirements.....	37
1.8.3 - Requirements Analysis.....	37
1.8.4 - Software Engineering Tools and Techniques.....	37
1.8.5 - Timeline.....	37
1.8.6 - Appendix A: UI Sketches.....	37
1.8.7 - Appendix B: Use cases.....	37

1.1 Executive Summary

Construction companies operate like well oiled machines when safety, communication, and productivity align, but many still depend on outdated paper-based systems to manage critical field operations. These manual processes are time-consuming, prone to data loss, and lack the real-time visibility that modern project teams require.

CrewConnect aims to solve these challenges by providing a unified digital platform that connects field crews, safety managers, and project engineers through one streamlined system.

CrewConnect combines two synchronized interfaces:

- A mobile application for field workers and foremen, enabling time tracking, safety checklists, equipment inspections, and daily production reporting.
- A web dashboard for management teams, offering live data analytics, crew oversight, and productivity insights.

This dual-interface approach bridges the gap between field activity and office reporting by ensuring every man-hour, inspection, and production metric is automatically recorded, analyzed, and visualized. By digitizing safety and production workflows, CrewConnect reduces administrative overhead, prevents costly errors, and enhances compliance with OSHA and company safety policies.

Built by individuals with hands-on construction experience, CrewConnect focuses on practicality, speed, and ease of use. It integrates geolocation tracking, digital signatures, and customizable forms into one cohesive system. The result is a platform designed not only to improve accountability but also to empower construction companies to make smarter, data-driven decisions about crew performance and project timelines.

CrewConnect's ultimate goal is to help construction companies modernize field operations by creating safer, more transparent, and more productive job sites.

1.2 Background and Technical Requirements

1.2.A – Similar ideas

A variety of applications address parts of this functionality, including BusyBusy, Procore, HCSS, FieldWire, and SiteMax. Each provides partial solutions, but none combine affordability, simplicity, and real-time data synchronization the way CrewConnect does.

Software	Strengths	Weaknesses	CrewConnect Advantage
Procore	Robust project management tools; document control	Expensive (5–6 figure annual costs); poor mobile UX	Affordable subscription model; tailored to field-level usability
HCSS	Heavy civil estimating and time tracking	High setup and training cost	Streamlined setup; integrated production + safety features
BusyBusy	Excellent GPS-based time tracking	Limited safety and production reporting	Adds real-time safety forms and production analytics
FieldWire	Clean interface, good document sharing	Lacks safety and time tracking	Combines all three core functions in one
SiteMax	Balanced all-in-one system	Weak data visualization and reporting tools	Live dashboards and analytics via Mapbox integration

The biggest initial problem that CrewConnect aims to solve that many of these programs have is cost. Procore is a commonly used commercial construction management dashboard, and its costs are significant. Depending on the exact features users are looking to use, and the budget of their project, users of Procore are looking at 5–6 figure costs for this program. HCSS is also very expensive, with lots of startup and training costs.

Another issue with these programs is that they lack a lot of the useful features we plan to add. Procore doesn't have any time tracking capabilities, nor does it have any sort of safety feature, no checklists. HCSS has time tracking, however it's very rudimentary with users just recording the amount of time they worked, not clocking in.

Our most similar competition is BusyBusy and Sitemax. They seem to have lots of the features we would like to implement, such as digital safety forms, materials, equipment, and time tracking. It does not have a good overview of the results of the safety forms that we would like to see however.

CrewConnect differentiates itself by focusing on usability, cost efficiency, and data visualization by offering small and medium contractors access to the same insights that large firms enjoy, but with a fraction of the complexity and expense.

1.2.B - Technologies to Leverage

CrewConnect uses a modern full-stack architecture designed for scalability and accessibility across devices.

Frontend

- Mobile Application (Expo + React Native):
The field app allows workers to clock in/out, fill out safety checklists, record quantities, and capture photos from their smartphones. Expo accelerates development and ensures cross-platform support for both iOS and Android.
- Web Dashboard (React + Mapbox GL JS):
The web interface allows managers and safety teams to view crew locations, production metrics, and reports in real time. Mapbox GL JS provides a geospatial visualization layer to display jobsite data interactively.

Backend

- Node.js (API Server):
Handles authentication, API requests, and business logic. Communicates between the frontend and database to maintain real-time data synchronization.

- Postgres (Database): A robust relational SQL database used to store user accounts, project records, time logs, and safety forms. While inherently schema-based, Postgres offers flexible data modeling through features like JSONB, allowing construction-related data structures to evolve without rigid migrations.

1.2.C - Software/Hardware Requirements

For Developers:

- Node.js (v18 or higher)
- Postgres database
- Expo CLI and React Native environment
- Mapbox API key for map visualization
- GitHub repository with CI/CD via GitHub Actions

For Users:

- Mobile Users: Any modern smartphone (Android 10+/iOS 13+), GPS enabled
- Web Users: Any desktop browser (Chrome, Edge, or Safari) with stable internet connection
- Backend Hosting: Cloud-based deployment on AWS EC2 or GCP VM
- Data Storage: Postgres cluster or equivalent

The lightweight tech stack ensures the system remains fast, responsive, and deployable across different environments without specialized hardware.

1.3 Requirements Analysis

1.3.1 System architecture

High-Level Description: CrewConnect is a mobile-first, AWS-hosted, Postgres-backed system with a Mapbox-powered geospatial layer. Field users interact through a React Native app; office/admins use a web interface. All data flows through a secured API layer running on AWS, which reads/writes to Postgres and stores media in S3.

1.3.1.1 Client Layer (Presentation)

1. Mobile App (Primary)
 - Framework: React Native (Expo)
 - Platforms: iOS, Android
 - Features:
 - Clock in/out
 - Cost code switching
 - Break/return from break
 - Safety/DVIR forms (with photos)
 - Mapbox map to show jobsite and "inside/outside geofence" status
 - Offline-first for time entries; background sync
2. Web / Admin Portal
 - Framework: React Native (Expo)
 - Users: admin, estimator, project manager
 - Features:
 - View crews and daily time
 - Approve "out of geofence" entries
 - Manage projects, cost codes, and geofences
 - Run exports/reports
3. Design Source
 - Figma as the single source of truth for UI (Material 3 style, our orange/neutral palette)
 - Screen → component mapping documented for developers

1.3.1.2. API / Application Layer

1. API Server
 - Runtime: Node.js
 - Framework: Express
 - Deployed on: AWS (Elastic Beanstalk or ECS/Fargate)

- Responsibilities:
 - Auth (JWT)
 - CRUD for users, projects, cost codes
 - Time entry creation and validation
 - Geofence validation endpoint
 - File-upload pre-signed URL generation for S3
 - Role-based access control (worker, foreman, admin)
- 2. Business Logic Modules
 - Timekeeping Module
 - Validates user state (not already clocked in)
 - Links time to project + cost code
 - Applies company rules (required geofence? required photo?)
 - Geofencing Module
 - Accepts {lat, lon} + projectId
 - Pulls geofence from Postgres
 - Runs point-in-polygon / point-in-radius
 - Returns decision: IN_RANGE, OUT_OF_RANGE, NEEDS_APPROVAL
 - Safety / DVIR Module
 - Stores form submissions
 - Links photo URLs from S3
 - Reporting Module (API-level)
 - Aggregates time by user, project, date
 - Supports CSV/PDF export (PDF can be done via Lambda)
- 3. Security
 - JWT access tokens
 - Passwords hashed (bcrypt/argon2)
 - CORS locked to mobile/web origins
 - Admin-only routes for project setup

1.3.1.3. Data Layer (Postgres)

- 1. Database
 - Postgres (managed)
 - Reason: flexible schemas for construction projects (jobsites, varying cost codes, dynamic forms)
- 2. Core Collections
 - users
 - name, email, role, companyId
 - projects

- name, location(s), active, customer
 - geofences
 - GeoJSON polygon or circle
 - linked to `projectId`
 - metadata: `createdBy`, `lastUpdated`
 - timelogs
 - `userId`, `projectId`, `costCodeId`
 - `clockIn`, `clockOut`, `duration`
 - `locationAtClockIn`
 - `geofenceStatus`
 - costcodes
 - `code`, `description`, `projectId` or `global`
 - safetyReports / dvir
 - form data JSON
 - S3 file links
 - equipment (optional for later)
 - `unit`, `hours`, `photos`
3. Indexes
- `timelogs`: index on `userId + date`
 - `geofences`: 2dsphere index for geospatial queries
 - `projects`: index on `companyId`
4. Data Access
- API uses a repository/service pattern → keeps business logic separate from Postgres queries
 - Validation done at the API level, not in the client

1.3.1.4. Geospatial / Mapbox Layer

1. Map Rendering (Client)
 - Mapbox RN SDK to show:
 - Current user location
 - Jobsite boundary (polygon)
 - Pin for project center
2. Geofence Source
 - Geofences stored in Postgres
 - Pulled to client on:
 - login
 - project change
 - manual refresh

3. Validation Flow

- User taps Clock In
- Client sends GPS + projectId → API
- API checks **geofences** collection → point-in-polygon
- API returns allow/deny/needs-approval
- Client shows banner (green = in, red = out)

4. Why server-side check?

- Prevents spoofing
- Allows audit trail
- Lets admin override

1.3.1.5. File / Media Handling

1. Storage

- AWS S3 for:
 - DVIR photos
 - Safety inspection images
 - Jobsite condition photos
- Files NOT stored in Postgres – only S3 URLs are stored

2. Upload Pattern

- Client requests pre-signed URL from API
- Client uploads directly to S3
- API stores metadata in Postgres

3. Access Control

- S3 buckets private
- URLs time-limited if needed

1.3.1.6. Infrastructure (AWS)

1. Compute

- Elastic Beanstalk or ECS/Fargate to run Node/Express API
- Optional AWS Lambda for:
 - Nightly reports
 - "Clocked in but didn't clock out" checks
 - Push notification jobs

2. Storage

- S3 (media, exports)
- Postgres Database (separate but in AWS region close to app)

3. Monitoring & Logging

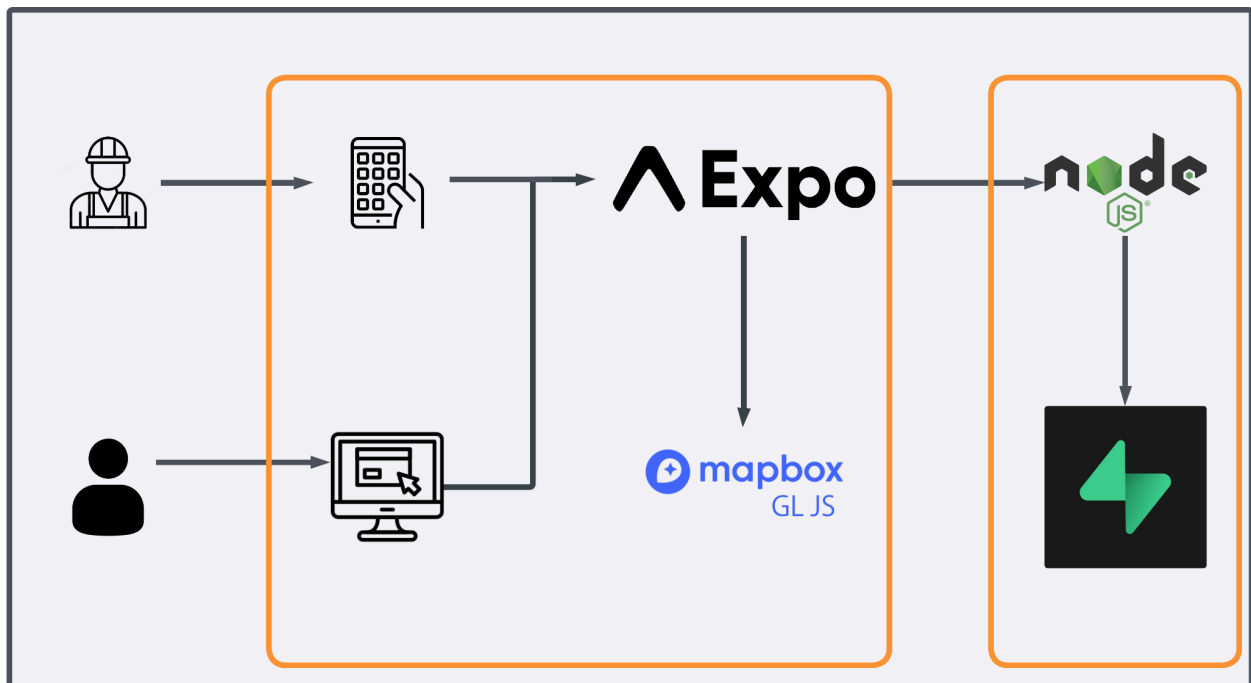
- CloudWatch for API logs and metrics

Crew Connect

- Error alerts to Slack/Email (optional)
4. CI/CD
- GitHub → build → deploy to AWS
 - Running tests on pull requests

1.3.1.7. Data Flow (End-to-End)

1. Worker opens app → logs in (JWT)
2. App fetches projects + geofences + cost codes
3. Worker taps Clock In → sends GPS + projectId → API
4. API validates geofence → writes to **timelogs** in Postgres database
5. If photo/DVIR → app uploads to S3 → API stores reference
6. Admin portal queries Postgres database → shows daily/weekly reports



1.3.2 Personnel

Crew Connect

Team Member	Role	Responsibility
Cody	Database administrator, Backend Developer	API creation, Database schema creation, backend logic, Supabase integration.
Liam	Frontend Web Developer	React UI/UX implementation for the web side of the project. Backend-web frontend integration.
Joel	Backend Developer	API calls, Data handling, backend-mobile frontend integration.
Spencer	Frontend App Developer	React Native mobile UI/UX implementation, navigation logic, backend work as needed.

1.3.3 System features

High Level Feature Grid				
Feature Area	Rank	Feature	Person Completing	Feature Description
Basic App UI	1	Authentication	Joel	Allows users to create an account, connect to their company, and reset their password. Role-based login (laborer, foreman, admin, etc.)
Clock	1	Clock In/Out System	Cody, Spencer	Start/stop timers tied to user, job, and cost code.
Clock	1	Cost Code Switching	Cody, Spencer	Easily switch between tasks (e.g., pipe install -> grading).
Clock	2	Geofencing Validation	Cody	Confirms the worker is on an approved jobsite before clock-in.
Clock	2	Offline Mode	Cody	Cashes time data and syncs automatically when back online.
App Navigation	1	App Screen	Spencer	Allows users to navigate to other features such as Observation page, or TimeCard
App Navigation	1	Bottom bar	Spencer	Bottom bar on every page allows user to navigate between the app page clock-in page and
Safety App	1	Observation Page	Spencer	This page is allows the user to fill out a safety observation form and submit it to the web UI
Safety App	1	DVIR Page	Joel	This page is allows the user to fill out a Driver Vehicle Inspection Report form and submit it to the web UI
App	3	Contact List	Spencer	This allows users to view a contact list for anyone who is working on the job you are clocked and allows the user to make calls
App	3	Log Books	Cody	This allows users to fill out logs for a specific task. Such as water truck logs or any log built in the web UI
App	3	Photo Galley	Cody	Allows users to see all the pictures they have taken for specific job and share them easily with anyone on the same job
App	1	Timecard section	Liam	Allows users to see their time on the job every day for the week and pay period. Allows users to input signatures from both the laborers and supervisors to approve the pay period

Crew Connect

				punches. Additionally allows users to request time off, and paid vacation time.
Website	1	Web Dashboard	Liam, Cody	Gives users a basic overview of various analytics; the most important things from the labor, materials, and projects sections will be shown here
Website	1	Web Safety Analytics	Spencer	Display various safety analytics of all the employees, and projects. Allow users to see more in depth analytics for specific safety checks, specific projects, and specific individuals within the company.
Website	1	Web Labor Analytics	Liam, Cody	Display the various labor analytics, showing costs and hours spent on labor for each project, over time.
Website	1	Web Projects Page	Liam, Cody	Display and allow users to edit, add, and remove company projects, and show materials analytics, along with the labor and safety analytics from the other pages.
Website	2	Web Home	Liam, Cody	Takes the place of the web dashboard for users who are not signed in. Display various marketing materials to show off our program.
Website	3	Web Support	Everyone	A page filled with tutorials to use our app and website, and ways to submit bug reports and contact us.
Website	3	Web Pricing	Cody	A page displaying the pricing for our program, alongside the billing system itself.
Website	2	Custom Safety Lists	Spencer	A way for the users to create custom safety checklists that can then be filled out on the mobile app, and displayed like the pre-made safety checks in the web safety analytics page.
Website	1	User accounts and roles	Cody	Allow users to create accounts, for both companies and employees within a company, and assign employees different roles/permissions according to their needs and requirements.
Website	3	Payroll Integration	Joel	Allow users to send timeclock information to payroll programs, so that the 2 systems can work together easily.
Website	3	Labor Export	Joel	Allow labor data to be exported to .xlsx, and/or in a format that other programs in this space utilize.
Website	3	Materials Export	Joel	Allow materials data to be exported to .xlsx, and/or in a format that other programs in this space utilize

1.4 Software Engineering Tools and Techniques

For our project, we plan to use a variety of software engineering tools and techniques. Some of these tools are ones we're already familiar with from past classes or personal projects, while others will be new to some or all members of our team. We'll prioritize tools that are free or offer free student versions, though we may use a few that require a small fee. In terms of techniques, we plan to incorporate a range of modern software engineering practices. The specific tools and techniques we intend to use are listed below.

1.4.1 Agile

We will follow an Agile-inspired workflow built around short, weekly iterations. Rather than planning the entire semester in detail upfront, we will plan 1-2 weeks ahead, allowing us to adapt quickly to new requirements, feedback, and evolving design decisions.

Our Agile plan includes:

- Weekly sprints with goals defined in GitLab.
- Sprint planning at the start of each week.
- Sprint review + retrospective at the end of each week.
- Continuous integration of completed tasks through merge requests.
- This structure provides flexibility while ensuring consistent progress and accountability.

1.4.2 Tools

1.4.2.1 IDEs

We will be using Visual Studio Code (VS Code) as our primary IDE for all development. VS Code offers many features that make it an excellent choice, including an intuitive version control system, powerful IntelliSense support, and a wide range of extensions that simplify the development process. It's also a tool we're already very familiar with, which makes it easy and efficient for us to use. Additionally, VS Code provides robust debugging tools for both React and React Native projects. The built-in Snippets feature further enhances productivity by allowing developers to insert boilerplate code with just a few keystrokes.

1.4.2.2 Main Libraries

- React / React Native – for frontend development (web + mobile).
- Node.js / TypeScript – for backend logic or API tooling.
- Supabase / PostgreSQL – for authentication, database, and storage services.
- shadcn/ui or Material UI (depending on project needs) – for consistent UI components.
- Expo – for mobile app development and testing.

1.4.2.3 Postman

Using Postman provides key development benefits by allowing rapid testing of API endpoints, easy debugging through clear request and response visibility, and efficient validation of authentication, parameters, and error handling. It also allows us to save and share request collections, ensuring consistent testing during the development process.

1.4.2.4 Testing Plans and Tools

We will incorporate testing throughout the development process that will include Unit Testing (Jest + React Testing Library for frontend components), Integration Testing for key data flows (authentication, API endpoints), and manual UI testing during weekly demos.

1.4.2.5 Bug Tracking

Utilizing the issue tracker in GitLab, the team will create issues pertaining to bugs as we test the program, or run into them. The issues will have a special label, indicating that the issue is a bug fix, and not a feature implementation. The issue should have a detailed explanation of expected behavior of the program, and the actual behavior, and a detailed set of instructions on how to replicate the bug.

1.4.2.6 Versioning

The team will use GitLab's branching to manage versions. The main branch of the repository will contain the most recent working version of the project, there should not be broken or buggy parts in the most recent version in the main branch. Branches off the main branch will contain the changes and additions to the existing version, and cannot be added back to the main branch without following our code review process (1.4.3)

1.4.2.7 Issue Tracking

We will use GitLab's issue tracking to manage tasks, bugs, and features. Each issue will include a clear description, priority level, and assigned team members to maintain visibility and accountability. GitLab's integration with commits and merge requests directly links development work to the issues they resolve, providing a clear and organized workflow.

1.4.3 Code Reviews, e.g., "entire team" vs "single signoff"

Every merge request must be reviewed by at least one other teammate before being merged. Reviews focus on correctness, readability, security concerns, and alignment with project architecture.

1.4.4 Documentation

Header comments on every function describing a general overview of what the function does, along with comments on any section of code that might require additional information to understand and/or work with. Comments should be written before anyone else has access to the code from the initial author, so as to reduce the chance of misunderstanding.

As major sections of the project are completed, information about the relevant sections should be written and/or updated in the wiki, so that the team and others can review the project and how each part is working.

1.4.5 Team Communication

We will use Discord for daily messaging and asynchronous updates, along with communicating deadlines for course assignments. Additionally we will utilize gitlabs issue board to communicate work that is being done, work that we plan on doing, and any other feature-specific info that may need to be communicated.

1.4.6 Team Meetings

We will maintain a consistent meeting schedule that consists of two meetings, one at 9 am on Mondays, and the other at 9:30 am on Wednesdays to report progress and unblock teammates as well as ad-hoc meetings as needed during critical milestones.

1.5 Timeline

Phase – Week	Spencer Perry	Joel Moffatt	Liam Coburn	Cody Mathews
Alpha – Week 1	Add functionality for users to upload a profile picture, and include additional fields for phone numbers and other personal details for user screen Add functionality	Update timecard to look like figma drawing	Add functionality for new users to sign up. The new user sign up creates a new company. A company will be in charge of managing/creating their employee's/user's accounts, so that will be implemented as	Build dashboard API endpoints – aggregate data for cost codes, budgets, labor hours by project.

Crew Connect

	for users to fill out after clock out form with safety and verifying that the time they have is correct.		well. Extend the login modal on the web to actually send password reset requests.	
Alpha – Week 2	Create a dedicated DVIR (Driver Vehicle Inspection Report) page that includes: Interactive checkboxes for all required inspection items Ability to attach photos for documentation and evidence Submission workflow that sends completed reports and images directly to the database Develop a DVIR sub-page on the web platform with tools for management and analysis, including:		Implement custom safety checklists that are created on the web side, then can be filled out on the mobile side, with analytics of those showing up on the web alongside the pre-made lists Dashboard Analytics content, showing an overview of safety, materials, labor, and project analytics on the web dashboard.	Create materials and equipment tracking endpoints. Add budget tables and CRUD operations.

	Visual graphs and analytics summarizing submitted DVIRs Filters for date range, vehicle, crew, or issue type Option to export DVIR data as CSV files for reporting, compliance, and recordkeeping			
Alpha – Week 3	Add forgot password submenu. That sends an email to you that allows you to reset the password Testing past features	/	Implement the web analytics for the safety, labor, project, and materials overview pages on the web side. Add account customization to the account icon on the website. Allow for profile pics, bios, and contact information.	Implement geofence table with polygon/GeoJSON storage. Build endpoint to save job site boundaries from web.
Alpha – Week 4	Make an Observation page, with check boxes and the ability to take pictures and submit	Deploy Alpha version to Render	Add the in-depth analytics for all of the overviews, (i.e. looking at the history and stats for specific	Create geofence validation endpoint – point-in-polygon check for mobile clock-in. Return geofence

	them to the database. Make Observation sub-page on the website with the ability to see graphs of filled out reports and make csv files from the data		individuals safety checks, or for specific projects), Replace home page temporary content with the real marketing materials. Update and 'beautify' features page	data to mobile for map display.
Alpha Milestone:	All analytics able to be seen on the web side, and all data that is needed for that is able to be recorded and sent from the mobile app			
Beta – Week 5	Implement a comprehensive Equipment page that allows users to: Log equipment hours Report equipment issues or problems View real-time availability status Track current equipment location		Add functionality to the support page, with bug reporting, suggestions/feedback, and tutorials/guides to the project. Implement automated testing for the web frontend	Build payroll export endpoint with pay cycle options (weekly, biweekly, semi-monthly, monthly). Generate downloadable CSV/Excel.
Beta – Week 6	Create a dedicated	Deploy updated backend and run	Continue automated front	Create DVIR (Driver Vehicle

Crew Connect

	Contacts page that displays all job-specific contacts, organized and grouped by role or personnel type.	integration tests	end testing on the web. Replace pricing page temp content with real content.	Inspection Report) table and endpoints. Support checklist submissions with photo references.
Beta – Week 7	Develop a centralized photo gallery within the app that allows teams to upload, view, and reference jobsite images.	Set up staging environment for testing	Implement actual pricing, payment processing, and subscription features, as well as the payment and free tiers.	Build safety checklist API – allow custom checklists created on web, filled out on mobile.
Beta – Week 8	Implement a mobile app module that stores day-to-day logs locally on the device, allowing users to quickly capture notes, photos, and updates even without an internet connection.	Optimize cloud deployment (logging, uptime)	Add chat features to the web interface. Add viewing reference and jobsite images in the projects pages.	Implement contacts table – job-specific contacts with roles. CRUD endpoints for contact management.
Beta Milestone:	All necessary features completed, web and app fully usable by “real” users.			
Final – Week 9	Modify the DVIR system so that web users can create and customize their own checklists	Automate builds and test suites in CI/CD	Implement exporting labor data to .csv files and potentially other useful file formats, so that	Create daily logs table and endpoints. Support notes, photos, and timestamps per

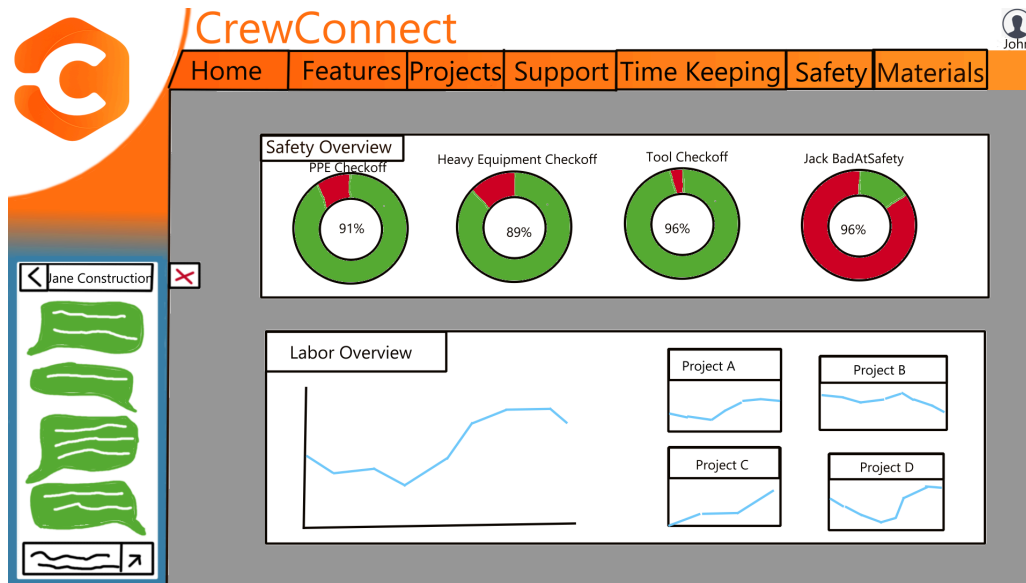
	and form items. This includes allowing them to edit existing lists, add new fields, reorder items, and define custom input types. All changes should automatically update the DVIR forms used in the mobile app.		users can easily re-use labor data for payroll programs	job.
Final – Week 10	Modify the Observations system so that web users can create and customize their own categories, checklists, and form fields. This includes the ability to edit existing items, add new fields, reorder sections, and define custom input types. All updates should automatically sync to the mobile app, ensuring field users see the latest Observation forms.	Ensure cloud monitoring and error recovery	Implement exporting materials cost data to .csv and other industry standard formats so that users can have copies of their data or utilize it with other programs	Dashboard refinements - additional analytics endpoints for safety, labor, materials reporting.

Crew Connect

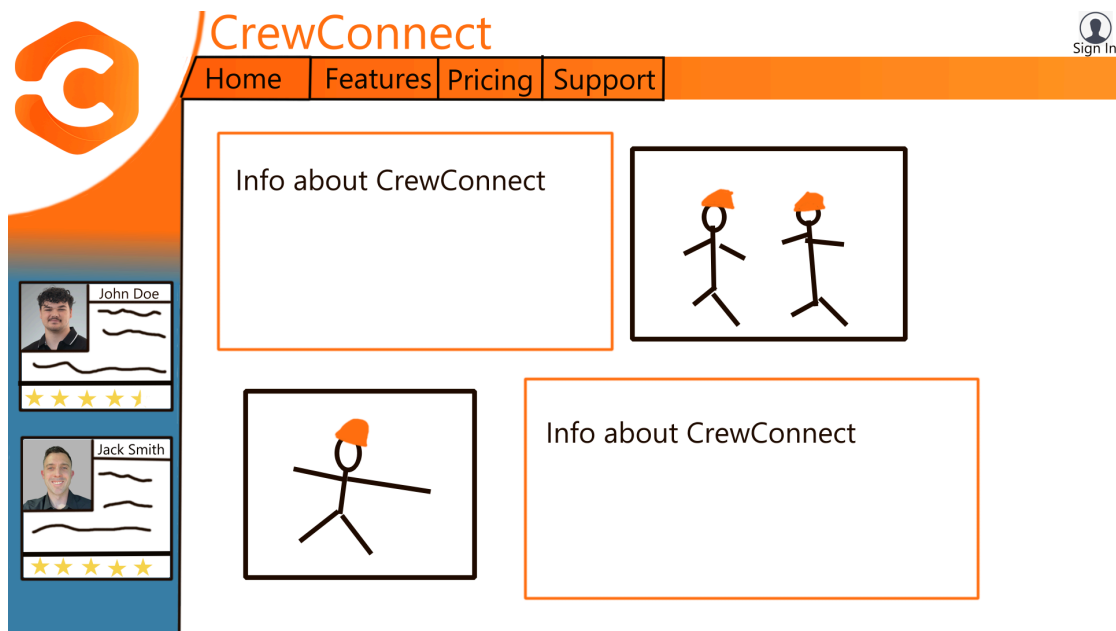
Final – Week 11	Test all Observation and DVIR checklist functionality to ensure custom fields, categories, and configurations work correctly across the web and mobile app.	Assist in deployment to production environment	Change the website to show a QR and link to download the app for users who visit the website on mobile.	Bug fixes, optimize queries, add any missing endpoints based on frontend needs.
Final – Week 12	Eliminate any remaining bugs, apply final polish, and complete any features that are still in progress.	Oversee production release and handoff	Squash any remaining bugs, polish, and wrap up any features that remain un-finished (hopefully none at this point)	Final testing, documentation updates, demo preparation.
Final Milestone:	CrewConnect deployed with the necessary features, and extra QOL features added.			

1.6 Appendix A: UI Sketches

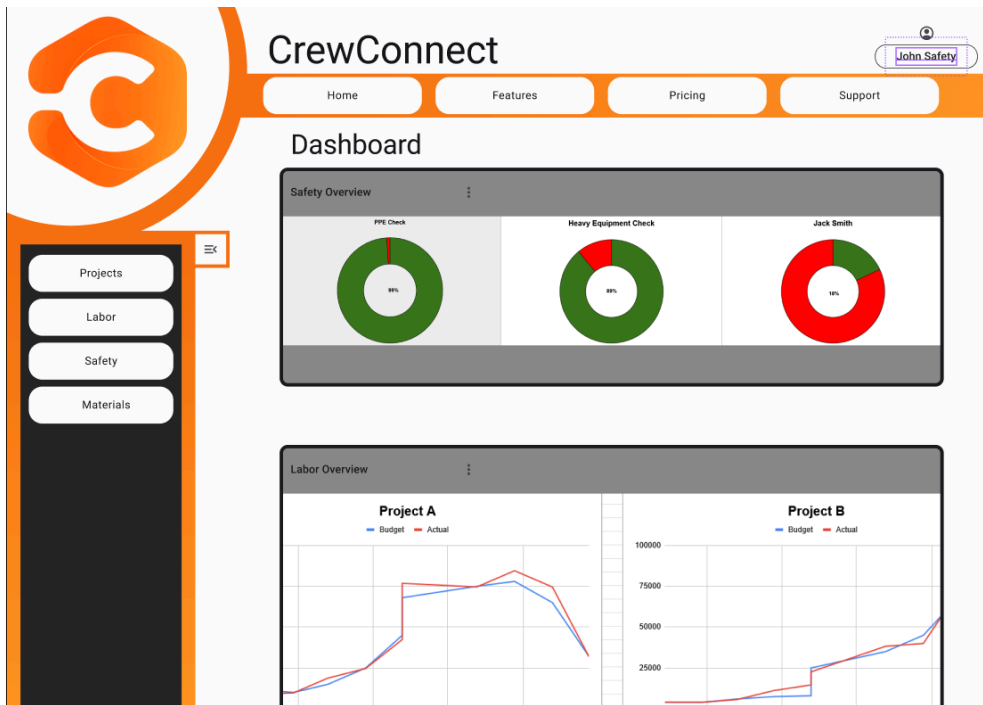
1.6.1 Web Dashboard Rough Sketch



1.6.2 Web Home Page Rough Sketch



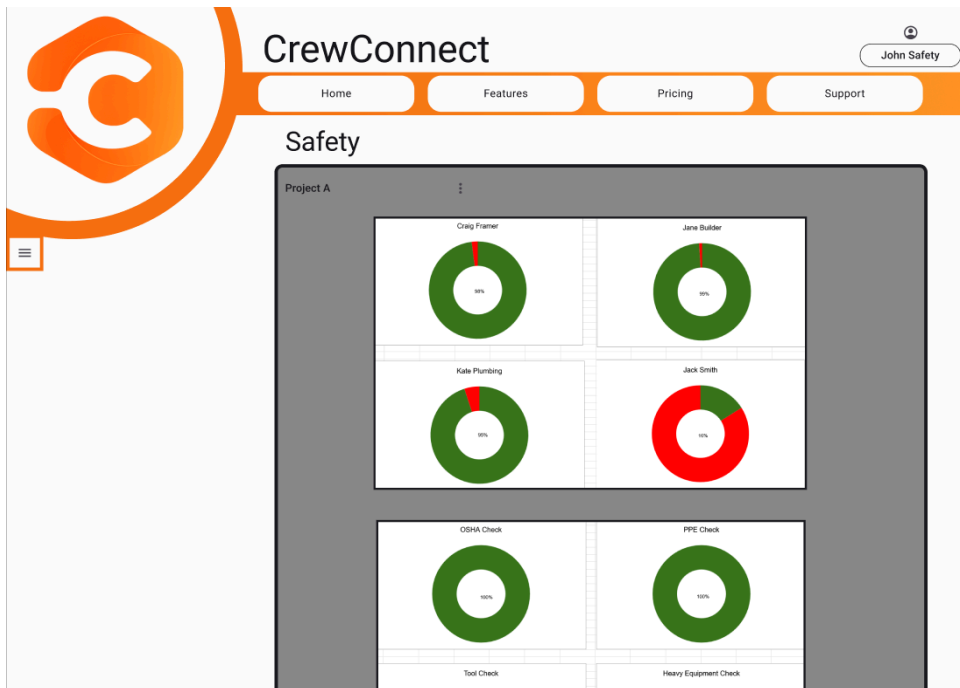
1.6.3 Web Dashboard



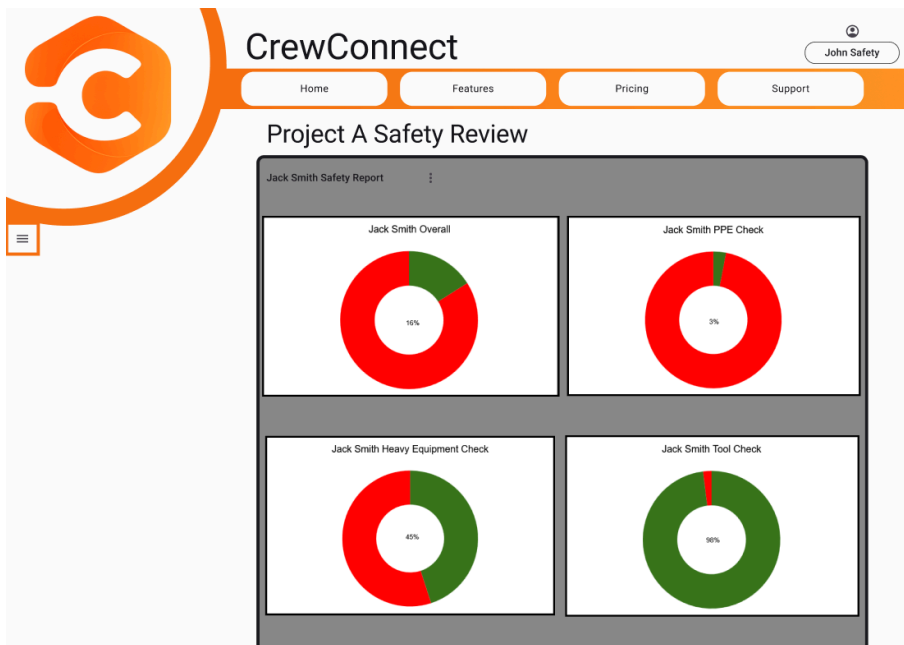
1.6.4 Web Safety Overview



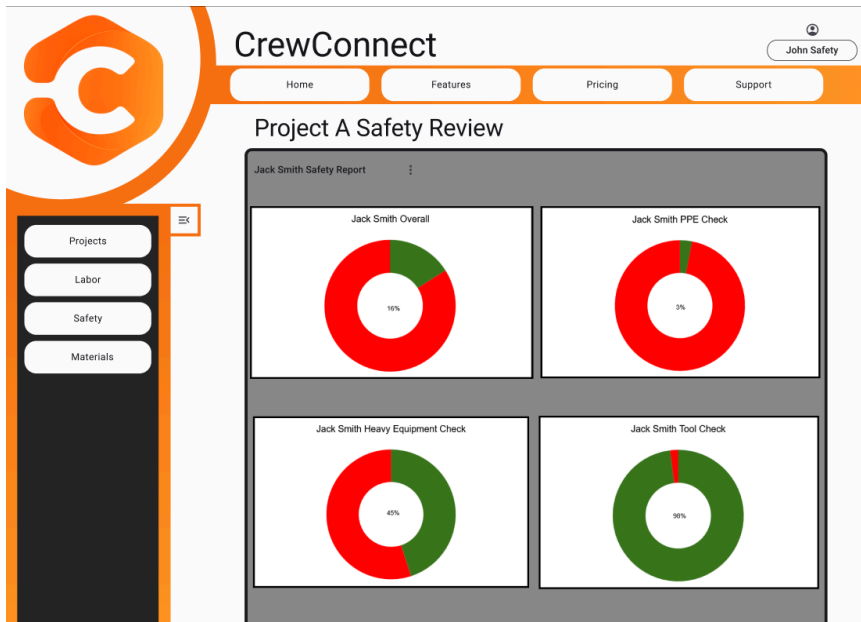
1.6.5 Web Safety Overview – No Sidebar



1.6.6 Team Member Safety Review – No Sidebar



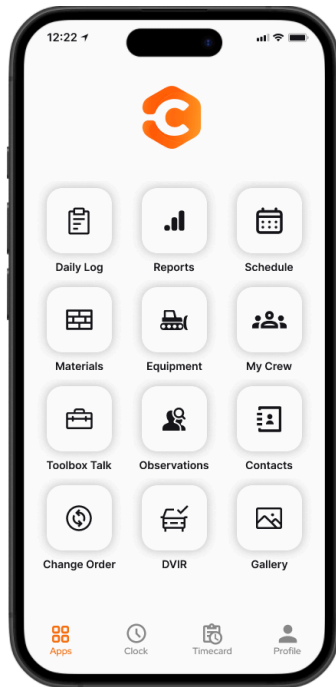
1.6.7 Team Member Safety Review



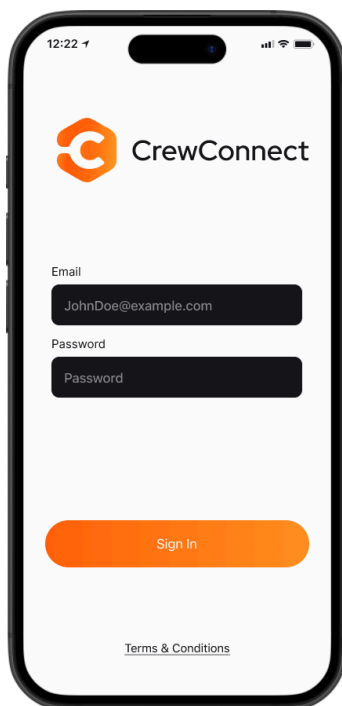
1.6.8 Web Labor Overview



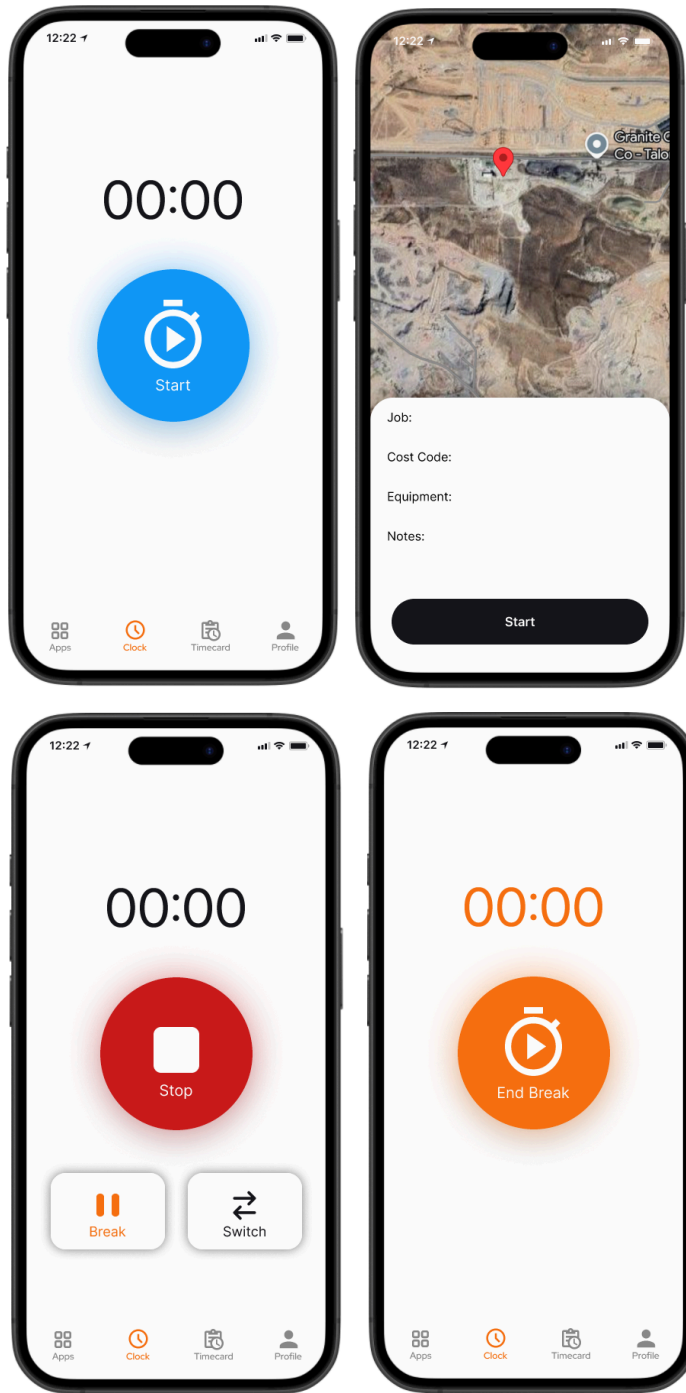
1.6.9 Mobile - App Screen



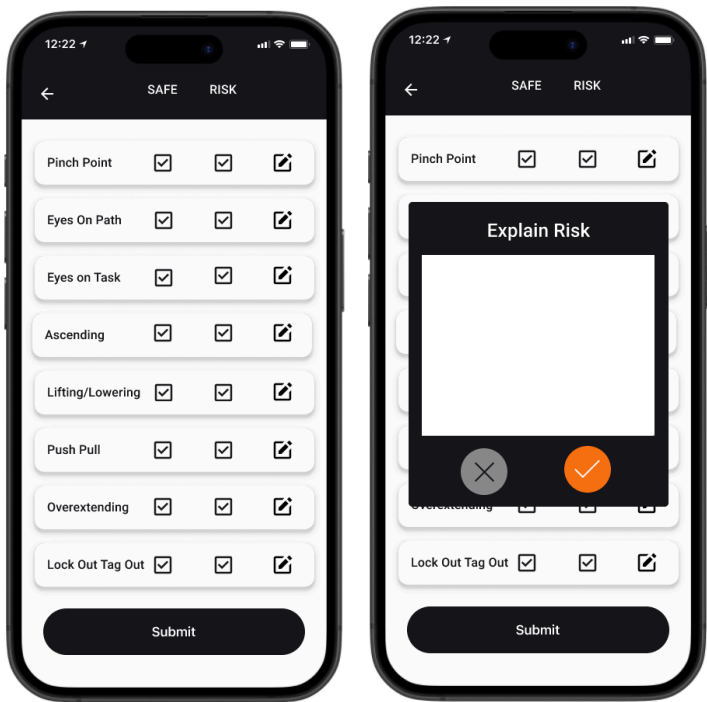
1.6.10 Mobile Login page



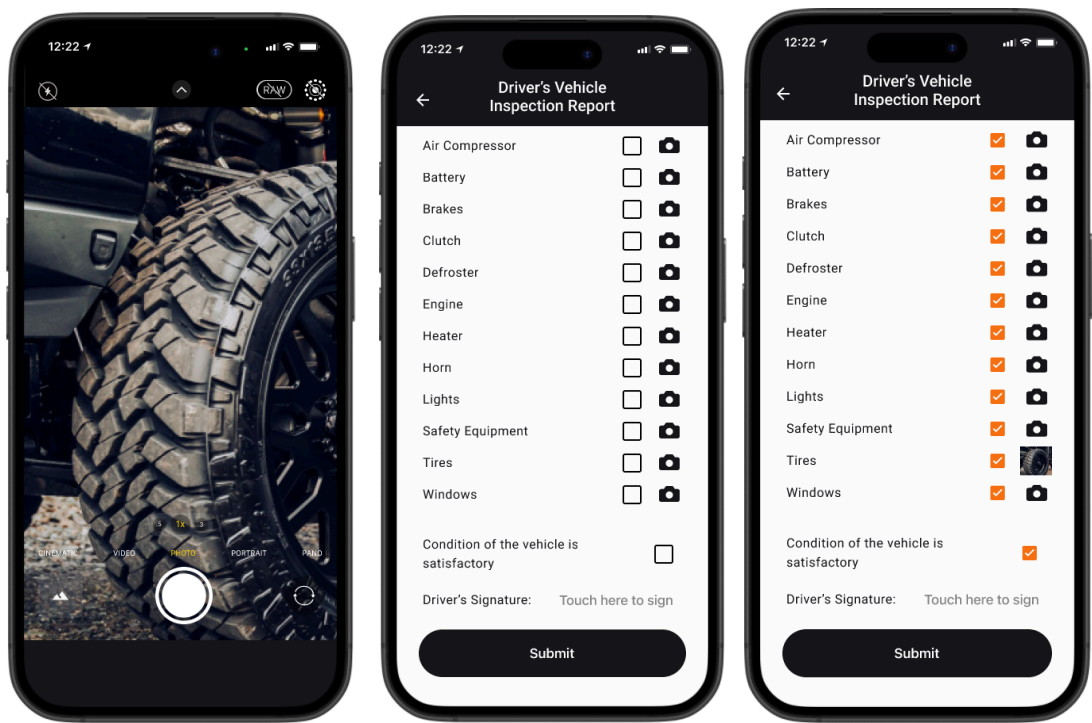
1.6.11 Laborer Clock In



1.6.12 Mobile - Observation Screen



1.6.13 Mobile - Pre Operation Screen



1.7 Appendix B: Use Cases

Number	Use Case 1
Title	Clock In, go on break, and Switch job
Preparer	Spencer Perry
Actor/User	Laborers
User Story	As a laborer, I want to clock in only when I'm at the jobsite, so that hours are accurate and verified.
Course of Events	1. Click the app on the home screen 2. Click clock in on Home Screen 3. Click the appropriate job click clock in on submenu
Exceptions/ Alternates	Use of go on break 1. Click go on break that is now on the home screen 2. Once break is completed click return from break Use of switch job 1. Click switch Jobs 2. Select the new job from the drop down 3. Click Save
Related UI	Mobile Login Page Laborer Clock in

Number	Use Case 2
Title	Office Safety Analyst reviews Jack Smith's Safety
Preparer	Liam Coburn
Actor/User	Office Safety Analyst
User Story	As the safety analyst in the office for the company, I want to go over the safety reports of all our projects and team members to make sure that we are being safe, and to correct any team members that are leading to poor safety outcomes.
Course of Events	1. Open up website dashboard 2. Navigate to the safety section via the sidebar 3. Close the sidebar 4. Navigate to an in depth view of Jack Smith's safety analytics by clicking on his graph 5. Review the safety graph data of Jack Smith
Exceptions/ Alternates	1. Don't Close the sidebar, then navigate to the jack smith safety page, which leaves the sidebar still viewable 2. Open up the sidebar after reviewing the team member's safety report to navigate to another screen
Related UI	Web Dashboard Web Safety Overview Web Safety Overview - No Sidebar Team Member Safety Review - No Sidebar Team Member Safety Review

Number	Use Case 3
Title	Doing a Safety Observation
Preparer	Spencer Perry
Actor/User	Laborers, Production Mangers
User Story	As a Laborer I want to fill out safety observations to promote safety culture and identify problems on the job so they can get fixed.
Course of Events	1. Click the app page button in the bottom right hand corner, this will navigate you to the app page. 2. On the app page click the icon with binoculars on it called observation 3. Once on the observation page go through the list of toggle switches selecting whether it is a risk or safe. Once done with all of the list click done and it will submit it to the safety team and return you to the apps page
Exceptions/ Alternates	None
Related UI	Mobile - App Screen Mobile - Observation Screen

Number	Use Case 4
Title	Doing a Pre operation
Preparer	Spencer Perry
Actor/User	Equipment operators
User Story	As an equipment operator, I need to verify that my equipment is in proper working condition, so that I can ensure it is safe to operate and free from potential issues before use. I do this by completing the pre operations.
Course of Events	4. Click the app page button in the bottom right hand corner, this will navigate you to the app page. 5. On the app page click the icon with dozer on it called Pre Ops 6. Once on the Pre operation page select the piece of equipment from the drop down, go through the list of toggle switches, text Fields selecting whether it is in good working order or not. Once done, hit done and it will return you to the page.
Exceptions/ Alternates	None
Related UI	Mobile - App Screen Mobile - Pre Operation Screen

Number	Use Case 5
Title	Analyze labor cost
Preparer	Joel Moffatt
Actor/User	Project Manager
User Story	As a project manager I need to review the current labor costs and compare them to what we budgeted. If I see that our actual labor costs are going over the budget or are likely to go over, I need to adjust staffing, change the scope, or request more budget.
Course of Events	1. Open up website dashboard 2. Navigate to the Labor section via the sidebar 3. Close the sidebar 4. Review the labor graph data of for each project
Exceptions/ Alternates	1. Click on Labor overview on home page to open labor section
Related UI	Web Dashboard Web Labor Overview

1.8 Appendix C: Revisions

1.8.1 – Executive Summary

- No changes made

1.8.2 – Background and Technical Requirements

- Updated the database from MongoDB to Postgres

1.8.3 – Requirements Analysis

- Adjusted personnel roles and feature responsibilities to more accurately reflect what we've done and what we plan on doing
- Adjusted system features table to be less pages
- Updated the database from MongoDB to Postgres

1.8.4 – Software Engineering Tools and Techniques

- Added more detail to the communication subsection to be more detailed about how the team communicates.

1.8.5 – Timeline

- Remade timeline section to more accurately describe our workflow for the next semester, incorporate the existing features from the prototype, and expand on ideas we've thought of to rework existing planned features.

1.8.6 – Appendix A: UI Sketches

- No changes made

1.8.7 – Appendix B: Use cases

- No changes made